

SPRINT 2 — PRODUCT BACKLOG

Real-Time Customer Support Ticket CRM using Apache Kafka

Node.js · Express.js · MongoDB · JWT · bcrypt · KafkaJS

Developer: **Arpita Maleth** | Reg No: **CU23BCA0011A** | Chanakya University | BCA Programme
Sprint 2 | Duration: **15 March 2026 – 28 March 2026** | Report Date: March 2026

Sprint KPI Summary

Total Sprints	Points Planned	Points Completed	Overall Status
2 (2-week cycle)	32 pts	32 pts (100%)	COMPLETE ✓
Stories Delivered	Sprint Duration	Sprint Status	Bugs Resolved
5 / 5	15 Mar – 28 Mar 2026	COMPLETE ✓	2 / 2 (Zero open)

Product Backlog — Sprint 2 Scope

The following table lists all user stories scoped for Sprint 2, covering the full Ticket Management System including CRUD operations, Ticket schema, Agent dashboard, and status tracking.

Story ID	Category	User Story / Task	Pts	Priority	Status
US-006	Database	Design the Ticket schema in MongoDB with fields: title, description, status (Open/In Progress/Closed), customerId, agentId, createdAt, updatedAt using Mongoose	6	High	✓ Done
US-007	Backend API	Implement POST /api/tickets — authenticated customers can create new support tickets; Kafka producer publishes ticket.created event on submission	6	High	✓ Done
US-008	Backend API	Implement GET /api/tickets — customers see only their own tickets; agents see all tickets; protected by authMiddleware and role check	6	High	✓ Done
US-009	Backend API	Implement PATCH /api/tickets/:id/status — agents can update ticket status (Open → In Progress → Closed); Kafka producer publishes ticket.updated event	8	High	✓ Done
US-010	Frontend	Build Agent Dashboard (dashboard.html) — displays all tickets in a table with status badges, fetches via GET /api/tickets using stored JWT token	6	Medium	✓ Done

Sprint Execution Detail

Sprint 2 (Wk 3–4) · E2 – Ticket Management System · Planned: 32 pts · Completed: 32 pts · 15 Mar 2026 → 28 Mar 2026

Story ID	Title	Pts	Actual	Completed	Status	Notes / Outcome
US-006	Ticket Schema (MongoDB)	6	6	17 Mar 2026	✓ Done	Mongoose Ticket model created. Fields: title, desc, status, customerId, agentId, timestamps.
US-007	Create Ticket API	6	6	19 Mar 2026	✓ Done	POST /api/tickets live. Auth middleware applied. Kafka producer publishes ticket.created event.

Story ID	Title	Pts	Actual	Completed	Status	Notes / Outcome
US-008	View Tickets API	6	6	21 Mar 2026	✓ Done	Role-based filtering: customer sees own tickets, agent sees all. JWT verified on every request.
US-009	Update Status API	8	8	25 Mar 2026	✓ Done	PATCH /api/tickets/:id/status restricted to agent role. Kafka publishes ticket.updated event.
US-010	Agent Dashboard UI	6	6	28 Mar 2026	✓ Done	dashboard.html renders all tickets via Fetch API. Status badges (Open/In Progress/Closed) styled.
TOTAL	All 5 Stories	32	32	28 Mar 2026	COMPLETE ✓	Zero carry-overs. 100% velocity achieved.

Retro Highlights: Ticket schema finalized on Day 3 — ahead of schedule. Role-based access control worked cleanly on first implementation using existing authMiddleware. Kafka producer events (ticket.created, ticket.updated) successfully tested with KafkaJS. Agent dashboard rendered real-time ticket data with correct role-gating on first integration test.	Impediments: Ticket status update initially allowed customers to change status (BUG-003, fixed Day 4). Dashboard Fetch API returned 403 due to missing role check in request header (BUG-004, fixed Day 6). Both bugs resolved within sprint — zero carry-overs.
--	---

Defect Log — Sprint 2

Bug ID	Description	Severity	Sprint	Status	Resolution
BUG-003	Customer role could update ticket status — no role guard on PATCH route	Medium	S2	Fixed	Added role === 'agent' check in ticketController.js before processing PATCH request.
BUG-004	Dashboard returned HTTP 403 — Authorization header missing from Fetch call in dashboard.js	Medium	S2	Fixed	Added headers: { Authorization: `Bearer \${token}` } to all dashboard.js Fetch requests.